université angers
FACULTÉ DES SCIENCES
Unité de formation et de recherche

L2 MI - 2025/2026
UE Développement Web
David Lesaint

CC JS : Session 1 - 1h30 - Sur machine

Téléchargez et décompressez l'archive déposée sur Moodle pour cet examen. Le dossier résultant contient différents fichiers à réutiliser ou à compléter. Vous testerez vos scripts avec le conteneur Docker et Firefox. A l'issue du temps imparti, archivez votre dossier et déposez l'archive sur Moodle.

Les exercices sont indépendants et peuvent être traités dans l'ordre de votre choix. Pour visualiser ce qui est attendu, consultez ce démonstrateur.

Exercice 1. Traitement de données

Complétez le fichier **mots.js** pour répondre aux questions qui suivent à partir du tableau de mots `words` qui y est initialisé. Testez-le en chargeant le fichier **mots.html**.

1. Créer et afficher le tableau associant à chaque mot de `words` sa taille (nombre de caractères). Afficher ensuite la taille maximum des mots du tableau.

2. Filtrer le tableau `words` pour ne conserver que les mots de 2 caractères ou plus et ne comportant ni `i`, ni `o`, ni `u`. Afficher le tableau résultant nommé `filtered` (51 mots).

3. Créer le tableau où chaque mot de `filtered` a été remplacé par le mot inversant ses caractères (p. ex. `abat` devient `taba`). Afficher le tableau résultant nommé `flipped`.

4. Tronquer les mots de 4 caractères du tableau `flipped` en supprimant leur premier caractère et en laissant inchangés les autres mots. Afficher le tableau résultant nommé `truncated`.

5. Trier le tableau `truncated` par longueur de mots croissante, et à longueur égale, en ordre alphabétique décroissant. Afficher le tableau résultant nommé `sorted`. La méthode `localeCompare` peut être utile.

6. Supprimer les doublons du tableau `sorted` pour ne conserver qu'un exemplaire de chaque mot, sans utiliser l'idiome `[...new Set(sorted)]`. Afficher le tableau résultant nommé `unique` (34 mots).

Exercice 2. Objets

Complétez le fichier **ensemble.js** pour répondre aux questions qui suivent. Testez-le en chargeant le fichier **ensemble.html** et en vous aidant de la console pour l'affichage des résultats.

1. Créez le constructeur `Ensemble` permettant de construire des objets modélisant chacun un ensemble d'entiers. Chaque objet est construit en communiquant un tableau, potentiellement vide, au constructeur. Le constructeur devra stocker ce tableau dans l'objet après avoir supprimé les éléments qui ne sont pas des entiers, supprimé les doublons et trié les entiers restants en ordre croissant. Pour supprimer les doublons, vous pouvez utiliser l'idiome `[...new Set(tableau)]` qui renverra le tableau `tableau` dédoublonné. Testez le constructeur en créant 4 `Ensembles` à partir des tableaux `[2,1,2,true,"0",NaN,1,3]`, `[]`, `[0,2,4,6,8]` et `[0,1,2,3,4,5,6,7,8,9]`.

2. Ajoutez une méthode `toText()` au prototype des objets `Ensemble`, qui renverra pour chaque objet sur lequel la méthode est invoquée, la chaîne de caractères au format $\{v_1, \ldots, , v_n\}$ où v_i est le i-ème entier de cet objet `Ensemble`. Testez la méthode sur les 4 objets créés précédemment.

3. Ajoutez la méthode `intersection(X,Y)` au prototype des objets d'`Ensemble` qui renverra l'objet `Ensemble` représentant l'intersection ensembliste des `Ensembles` `X` et `Y` passés en arguments. Vérifiez sur les objets de tests que `[0,1,2,3,4,5,6,7,8,9]` est le neutre et `[]` l'absorbant pour l'intersection.

4. Ajoutez la méthode `inter(Y)` au prototype des objets `Ensemble` qui modifiera l'objet `Ensemble` sur lequel elle est invoquée par son intersection avec l'`Ensemble` `Y`. L'implémentation de la méthode devra appeler la méthode développée en question précédente. Implémentez quelques tests sur les objets créés précédemment.

5. Créez un objet littéral `primes` ayant la propriété dénommée `tab` de valeur `[1,2,3,5,7]`. Modifiez le prototype de cet objet pour pouvoir invoquer la méthode `toText()`. Testez en console.

Exercice 3. API DOM

Le fichier **caesar.html** importe le fichier **caesar.js**. Complétez ce dernier, sans modifier le premier, pour répondre aux questions qui suivent. **caesar.html** contient un tableau HTML dont chaque case contient un caractère du mot codé *"Yoxsl!"*. Ces caractères ont été obtenus grâce au chiffrement par décalage, et le but de cet exercice est de déchiffrer ce message. Afin de décoder les messages, vous devrez afficher tous les décalages possibles dans un tableau. Vous trouverez en commentaire dans le fichier **caesar.js** les résultats des premières questions pour vous aidez. La Figure 1 illustre le fonctionnement du chiffrement par décalage. Les Figures 2 et 3 sont des captures d'écrans des premières lignes du tableau obtenu à la Question 3 et à la Question 6.

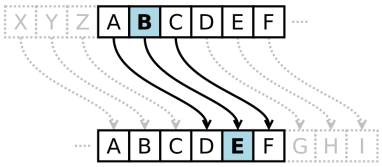

FIGURE 1 – Illustration du chiffrement par décalage de 3.

1. En utilisant l'API DOM, sélectionnez le tableau HTML contenant le message codé et stockez les caractères dans un tableau JavaScript.

2. À partir du tableau de caractères récupéré à la Question 1, générez une ligne de tableau contenant un élément `th` vide suivi d'un élément `th` pour chaque caractère du mot.

3. Complétez le tableau HTML en ajoutant une ligne par décalage (il y aura donc 25 lignes). Ainsi, chaque ligne sera composée d'un élément `th` contenant le décalage (un entier allant de 1 à 25), suivi des caractères du mot auquel on aura appliqué le décalage. Vous pouvez utiliser la fonction `decalageAsciiChar` qui prend en paramètre un caractère et son décalage et renvoie le caractère obtenu en appliquant le décalage.

4. Sans modifier le code précédent, supprimez la dernière colonne du tableau.

5. Le mot décodé est "Bravo". Sans sélectionner la ligne contenant ce mot par son indice, parcourez le tableau HTML pour trouver cette ligne et ajoutez la classe "highlight" à tous ses éléments `td`.

6. Appliquez la police de caractères `cursive` à tous éléments de balise `th` de la page. Vous ajouterez également la police `serif` par défaut.

	Y	o	x	s	l	!
1	Z	p	y	t	m	V
2	A	q	z	u	n	W
3	B	r	a	v	o	X
4	C	s	b	w	p	Y
5	D	t	c	x	q	Z

FIGURE 2 – Résultat attendu à la Question 3.

	Y	o	x	s	l
1	Z	p	y	t	m
2	A	q	z	u	n
3	B	r	a	v	o
4	C	s	b	w	p
5	D	t	c	x	q

FIGURE 3 – Résultat attendu à la Question 6.